Chapitre 1

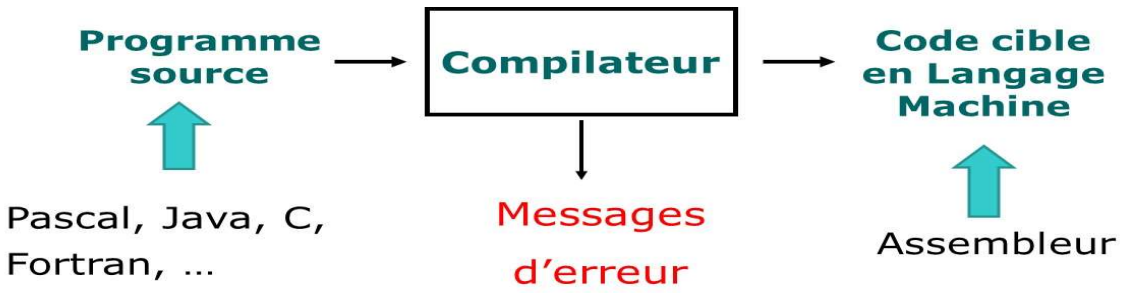
PRESENTATION GENERALE D'UN COMPILATEUR

1. Introduction

La compilation constitue la première étape de la chaîne de production de programmes.

2. Définition

La **compilation** permet la traduction d'un programme dit ***programme source*** écrit le plus souvent en langage de haut niveau vers un programme dit ***programme objet*** qui est soit directement le langage machine, soit le langage d'assemblage.



3. Les principes fondamentaux de la compilation

- Le compilateur doit conserver le sens du programme compilé.
- Le compilateur doit améliorer le code.

3. 1. Les propriétés de la compilation

- Le code produit doit être efficace (rapidité, mémoire).
- Le choix d'informations retournées en cas d'erreurs doit être judicieusement fait.
- La rapidité de la compilation doit être prise en compte.

3. 2. La structure d'un compilateur

Un compilateur peut être subdivisé en différentes phases :

3.2.1. Analyse lexicale

Reconnaissance des mots du langage ie. appréhension du vocabulaire : Elle lit la chaîne de caractères d'entrée, caractère par caractère, et la transforme en une suite de mots ou unités lexicales ;

3.2.2. Analyse syntaxique

Vérification de la syntaxe ie. appréhension de la grammaire : Elle vérifie si la suite d'unités lexicales reconnues par l'analyseur lexical est légale. L'analyse syntaxique dérive d'une structure hiérarchique d'unités lexicales qui peut être représentée par des arbres abstraits.

a. Analyse syntaxique ascendante : L'analyseur cherche, à partir de la phrase et à l'aide des règles de la grammaire, à remonter à l'élément initial (nous montons des feuilles vers la racine)

b. Analyse syntaxique descendante : L'analyseur cherche, à partir de l'élément initial et à l'aide des règles de la grammaire, à former la phrase donnée (nous descendons de la racine vers les feuilles)

3.2.3. Analyse sémantique

Vérification de la sémantique ie. appréhension du sens **:** Elle contrôle si le programme source a un sens. L'un des rôles importants de l'analyse sémantique est, en particulier, le contrôle de type : elle vérifie si les opérandes de chaque opérateur sont conformes aux spécifications du langage source (opérations arithmétiques entre nombres entiers, ou réels...).

3.2.4. La production du code objet

Le but final d'un compilateur est la production d'un programme objet.

3.2.5. L'optimisation du code

Le code produit par le compilateur doit être performant (ie. rapide à l'exécution et peu coûteux en place mémoire)

3.2.6. Le traitement des erreurs

Un compilateur doit détecter les erreurs lexicales, syntaxiques ou sémantiques, donner des diagnostics clairs et précis et être capable de poursuivre l'analyse du programme source.

Exemple d'erreurs

• Lexicales, comme l'écriture erronée d'un identificateur, d'un mot clé ou d'un opérateur ;
• Syntaxiques, comme une expression mathématique mal parenthésée ;
• Sémantiques, comme un opérateur appliqué à un opérande incompatible ;
• Logiques, comme un appel récursif infini.